ТЕХНИЧЕСКИЙ СРАВНИТЕЛЬНЫЙ ОТЧЁТ

RGS.Center и GDRMonitor

Схема подключения к портам, очереди команд, потоки чтения и записи

Архитектурное соответствие
80-85%

Фактическая готовность протоколов
55-65%

Основание: фактический код RGS.Center и актуальная локальная версия GDRM_monitor_latest. Дата анализа: 29 июля 2026.

1. Итоговая оценка

RGS.Center в высокой степени воспроизводит ключевой принцип GDRMonitor: одно устройство владеет подключением, передача и приём выполняются раздельными потоками, команды проходят через очередь, а входящие байты накапливаются и преобразуются в законченные кадры.

Важно: RGS.Center не копирует устаревшие механизмы GDRMonitor. Несинхронизированные коллекции, циклический Thread.sleep и Thread.stop заменены блокирующими очередями, lock, interrupt и контролируемым завершением ExecutorService.

Принцип GDRMonitor	Реализация RGS.Center	Оценка
Объект владеет физическим портом	AbstractDeviceConnection и наследники COM/LAN	**100%**
Отдельные потоки чтения и записи	RX/TX в PeripheralDeviceWorker	**100%**
Очередь обычных и приоритетных команд	Ограниченный LinkedBlockingDeque	**100%**
Кольцевой приёмный буфер	Защищённый ByteRingBuffer	**100%**
Выделение пакетов из потока байтов	LineFrameDecoder и FixedLengthFrameDecoder	**90%**
Последовательность команда - ответ	exchangeLock, TX/RX-очереди и timeout	**85%**
Автомат состояний соединения	Состояния worker есть, полного reconnect-автомата нет	**55%**
Готовые протоколы оборудования	Сканер и LAN-ping; весы и SHP ожидают протоколы	**50%**

Вывод

По схеме потоков и очередей соответствие составляет 80-85%. По полноте прикладных протоколов и автоматике восстановления связи проект пока находится на уровне 55-65%.

2. Сопоставление потоков и данных

Схема GDRMonitor

В GDRMonitor низкоуровневый передатчик TH_TraL берёт команды из LinkedList и пишет их в SerialPort. Событийный PortReader помещает входящие байты в CircleBuffer. Поток TH_RecL выделяет законченные ответы, а TH_RecH выполняет прикладной разбор. Отдельный TH_CONN контролирует состояния и watchdog.

```
SerialPort listener -> CircleBuffer -> TH_RecL -> ReqAndResp -> TH_RecH
Application command -> LinkedList -> TH_TraL -> SerialPort
```

Схема RGS.Center

В RGS.Center PeripheralDeviceWorker запускает два выделенных потока. TX блокируется на очереди команд и последовательно вызывает write. RX блокируется на чтении транспорта, записывает байты в ByteRingBuffer, выделяет кадры через FrameDecoder и публикует их в frameQueue.

```
Device service -> bounded command queue -> TX thread -> DeviceConnection
DeviceConnection -> RX thread -> ByteRingBuffer -> FrameDecoder -> frameQueue
```

Функциональные соответствия

- addBytesToSendNormally в GDRMonitor соответствует submit в RGS.Center.
- addBytesToSendFirst соответствует submitFirst и приоритетной части deque.
- CircleBuffer соответствует ByteRingBuffer.
- TH_TraL соответствует transmitLoop.
- TH_RecL и часть PortReader соответствуют receiveLoop и FrameDecoder.
- Флаг ожидания ответа соответствует exchangeLock и exchange с timeout.

Отличие модели потоков

GDRMonitor разделяет низкоуровневое получение кадра и прикладной разбор на два потока. В RGS.Center FrameDecoder работает внутри RX-потока, а прикладной сервис получает уже готовый кадр. Для простых протоколов это достаточно. Для тяжёлого двоичного разбора потребуется отдельный processing worker.

3. Что в RGS.Center реализовано безопаснее

Область	GDRMonitor	RGS.Center
Очередь команд	LinkedList без защиты	LinkedBlockingDeque с ограничением
Ожидание команды	Проверка очереди и Thread.sleep	Блокирующий take
Приёмный буфер	volatile-индексы и частичная synchronized-защита	ReentrantLock на всех операциях
Переполнение	Переход буфера в ошибочное состояние	Ограничения, счётчики и контролируемое вытеснение
Остановка	Флаги, join и местами Thread.stop	interrupt, close, shutdownNow, awaitTermination
Граница потоков	Общие изменяемые объекты	Копирование массивов и immutable snapshot

Практический вывод

Точное копирование потоковой реализации GDRMonitor не требуется. RGS.Center уже сохраняет нужную схему ответственности, но устраняет гонки данных, активное ожидание и небезопасное принудительное завершение.

Автоматический запуск

DeviceConnectionManager обнаруживает все ManagedDeviceConnection в Spring-контексте, запускает их при старте и закрывает при остановке. Ошибка одного устройства не блокирует остальные подключения.

4. Неполное соответствие и дальнейшие работы

Что ещё отсутствует

- Полный автомат DISCONNECTED - CONNECTING - CONNECTED - DEGRADED - RECONNECTING.
- Автоматический reconnect с задержкой и ограничением частоты попыток.
- Watchdog отсутствия входящих данных и переход к переподключению.
- Идентификатор команды для точного сопоставления ответа при сложных протоколах.
- Повтор команды после timeout и настраиваемая политика retry.
- Реальные TCP/UDP-адаптеры: текущий LAN-слой обслуживает reachability ping.
- Прикладные COM-протоколы весов и SHP.
- Аппаратные интеграционные тесты с реальным COMX и виртуальной парой портов.

Ограничения текущей оценки

Сканер уже использует постоянный RX-worker, но является преимущественно пассивным устройством и пока не демонстрирует полноценный двусторонний протокол. Весы и SHP имеют только резервные настройки пустых COM-портов. Поэтому высокая архитектурная оценка не означает полную готовность всего оборудования.

Рекомендуемый порядок достижения 90-95%

- Добавить типизированный DeviceCommand с correlationId, timeout и ожидаемым типом ответа.
- Ввести единый автомат состояния соединения и reconnect/backoff.
- Реализовать первый полноценный двунаправленный COM-адаптер.
- Подключить весы и SHP после фиксации их framing и команд.
- Добавить стендовые тесты COM и fake TCP server для LAN.

Заключение: RGS.Center правильно перенёс принцип GDRMonitor и уже превосходит его по безопасности многопоточности. Главная оставшаяся работа относится не к очередям и Thread, а к протоколам устройств, watchdog и автоматическому восстановлению связи.

Проверенные исходники

RGS.Center: center/src/main/java/com/rgs/center/peripheral/PeripheralDeviceWorker.java

RGS.Center: center/src/main/java/com/rgs/center/peripheral/PeripheralCommandQueue.java

RGS.Center: center/src/main/java/com/rgs/center/peripheral/ByteRingBuffer.java

RGS.Center: center/src/main/java/com/rgs/center/connection/AbstractDeviceConnection.java

GDRMonitor: GDRM_monitor_latest/src/com/bsi/tsw/gdrm/connection/GDRMConnection.java

GDRMonitor: GDRM_monitor_latest/src/com/bsi/tsw/gdrm/connection/GDRMConnectorTransmitter.java

GDRMonitor: GDRM_monitor_latest/src/com/bsi/tsw/gdrm/connection/GDRMConnectorReceiver.java

GDRMonitor: GDRM_monitor_latest/src/com/bsi/tsw/gdrm/connection/CircleBuffer.java